

CSE234 Week 7: Callback Mechanisms

This week, we will discuss the mechanisms provided in C++ for callbacks. A callback is a function that is passed to other functions as an argument. This allows one function to call different functions depending on its argument. The STL algorithms library depends heavily on callbacks, so we must discuss them before moving on to the algorithms library. C++ provides three mechanisms for callbacks: function pointers, function objects (functors), and lambda expressions.

Function Pointers

To declare a function pointer, you need to provide the return type, name of the function pointer variable (which has the pointer asterisk in front of it and is wrapped in parentheses), and the list of types expected for the parentheses. In the following example, the third parameter of `addOrSubtract()` is a function pointer that returns an integer, and expects two integers as its parameters. The name of the function pointer is `operation`. It can be used directly as a function, as seen in the return statement of `addOrSubtract()`.

```
#include <assert.h>
// addOrSubtract takes two integers, x and y, that should be
// added or subtracted, and a function pointer, operation
// that performs the operation
int addOrSubtract(int x, int y, int (*operation)(int, int)) {
    return operation(x, y);
}

int add(int x, int y) {
    return x + y;
}

int subtract(int x, int y) {
    return x - y;
}
```

```

int main() {
    // to use add, we can use an ampersand
    // on add's name to pass its address
    int additionResult = addOrSubtract(2, 2, &add);
    assert(additionResult == 4);
    // however, the ampersand is optional
    // the compiler always knows to pass the address of a function
    int subtractionResult = addOrSubtract(3, 2, subtract);
    assert(subtractionResult == 1);

    return 0;
}

```

C++ 11 added a header `<functional>` that includes the `std::function` type. `std::function` is a generic type which takes a nameless function signature as its template parameter. Below, the `addOrSubtract` function has been modified to use `std::function` as its type:

```

// need to include the functional header
#include <functional>
// std::function is a generic type
// the template argument is the return type, followed by
// the parameter list's data types in parentheses
// so, operation is a function that returns an int
// and expects two ints as parameters
int addOrSubtract(int x, int y,
    std::function<int(int,int)> operation
) {
    return operation(x, y);
}
// the rest of the program would be the same

```

Any function with an `std::function` parameter can also accept a functor or lambda as an argument, provided it matches the signature. For the most part, using `std::function` has replaced the need for function pointer types in

Functors

A functor is a class or struct that overloads the function call operator, i.e., it implements `operator ()`. Typically, functors only implement this operator, although they can implement other functions as well. The advantage of functors is that they can maintain state via their member variables. The below example implements a functor to compute the

average of values in an array:

```
#include <iostream>
#include <array>

// Average is a functor since it implements the call operator
struct Average {
    // Constructor to init fields to 0
    Average() {
        sum = 0;
        count = 0;
    }

    // operator() adds the value passed to it to the sum,
    // and increments the count
    void operator()(in value) {
        sum += value;
        ++count;
    }

    // property to return computed average
    double average() const {
        if (count == 0) return 0;
        else return static_cast<double>(sum) / count;
    }
private:
    int sum;
    unsigned count;
};

int main() {
    std::array<int, 7> values = { 2, 6, 24, 76, 3, 67, 5 };
    // declare an instance of the functor functor
    Average average;

    for (int x: values) {
        // overloading operator() allows us to use the object
        // like it was a function by calling it
        average(x);
    }

    std::cout << "Average = " << average.average() << std::endl;

    return 0;
}
```

Here is an example of a function that finds the maximum value in an array. It expects an `std::function` as a parameter, and we'll declare a functor to use as the argument.

```

#include <array>
#include <functional>
#include <iostream>

std::size_t findIndexOfMax(
    const std::array<int, 7> &arr,
    std::function<int(int, int)> compare
) {
    int currentMaxIndex = 0;
    for (std::size_t i = 1; i < arr.size(); ++i) {
        if (compare(arr[currentMaxIndex], arr[i]) < 0) {
            currentMaxIndex = i;
        }
    }
}

// returns -1 if x < y
// returns 0 if x == y
// returns 1 if x > y
struct Comparator {
    int operator(int x, int y) {
        if (x < y) return -1;
        else if (x > y) return 1;
        else return 0;
    }
};

int main() {
    std::array<int, 7> values = { 2, 6, 24, 76, 3, 67, 5 };
    // passing a Comparator directly by invoking its constructor
    std::size_t maxIndex = findIndexOfMax(values, Comparator());
    std::cout << "Max index: " << maxIndex << "\n";
    std::cout << "Max value: " << values[maxIndex] << "\n";

    return 0;
}

```

Lambda Expressions

Also called "lambdas" or "anonymous functions", lambda expressions allow you to put a function anywhere you would put an expression, such as the right-hand side of an equals sign or as an argument to a function. Lambdas consist of three parts: a capture, a parameter list, and a body. Below is the basic syntax of a lambda:

```
auto lambda = [capture1, capture2](type1 param1, type2 param2) {
    statements;
};
```

The capture is denoted with square brackets, and allows you to pass local variables to the lambda's scope. If the lambda needs to modify a variable it captures, the lambda needs to be marked as mutable and the variable must be passed by reference using the ampersand (&). Otherwise, passing captures by reference is treated as a const reference. The parameter list is like a typical parameter list for a normal function. It has the types and names of the expected arguments to the lambda. Finally, the body contains the statements that will be executed when the lambda is called. The body has its own local scope like any other function. Any statements that can be placed in the body of a function can be put in a lambda's body as well. Remember that the body of the lambda cannot access any local variables from the block it is called in unless they are passed via the capture. Below is an example of using a lambda to compute the average of the same array from the functor example:

```
#include <iostream>
#include <array>

int main() {
    std::array<int, 7> values = { 2, 6, 24, 76, 3, 67, 5 };

    // we need to declare variables for the sum and count locally now
    int count = 0;
    int sum = 0;

    // we can either use auto type or std::function<> to declare a
    // lambda "variable"
    // to allow the lambda to modify the values of count and sum,
    // they must be passed by reference and the lambda
    // needs to be marked as mutable
    auto averager = [&count, &sum] (int value) mutable {
        sum += value;
        ++count;
    }; // since this is part of a declaration, we need a semicolon

    for (int x: values) {
        averager(x);
    }
```

```
std::cout << "Average = " << static_cast<double>(sum) / count << s  
  
return 0;  
}
```

Note: It is very easy to forget to put a semicolon at the end of a lambda expression, so pay attention.

The return type of a lambda can be explicitly provided as well. Often, the return type of a lambda can be deduced by the compiler, but sometimes it may be required. For example, if one path of the lambda returns a string literal and another returns an `std::string`, the compiler will not choose between `const char*` and `std::string` for its return type:

```
// need to specify the return type since the compiler will not  
// choose between std::string and C strings on its own  
auto cStringOrString = [](bool returnStdString) -> std::string {  
    if (returnStdString) {  
        std::string str = "hello world\n";  
        // returning an std::string  
        return str;  
    }  
    else {  
        // returning a string literal (a C string)  
        return "hello world\n";  
    }  
};
```

Terminology

There are a few common categories of callbacks used by the STL algorithms. Here, we'll define them for future reference when discussing the algorithms library.

- Unary Function - A void function that takes one parameter.
- Unary Predicate - A function that returns a boolean and expects one parameter.
- Binary Predicate - A function that returns a boolean and expects two parameters.
- Binary Operation - A function that expects two parameters of the same types, and returns the type of its parameters.

Now, let's look at some examples of each. All the examples are using lambdas to define the callbacks, since that is the most common way to define a callback after C++11 was released.

Example Unary Callbacks

```
// this lambda doubles an int
auto doubler = [](int &x) { x *= 2; };
// this lambda prints a double with 2 decimal points
auto printDouble = [](double x) {
    std::cout << std::fixed << std::setprecision(2) << x;
};
```

Example Unary Predicate Callbacks

```
// this lambda returns true if a number is even
auto isEven = [](int x) { return x % 2 == 0; }
// this lambda returns true if a character is
// an uppercase letter
auto isUpper = [](char c) {
    return c >= 'A' && c <= 'Z';
};
```

Example Binary Predicate Callbacks

```
// returns true if the first parameter is greater
// than the second parameter
auto isGreater = [](double x, double y) {
    return x > y;
};

// returns true if the first parameter is
// divisible by the second
auto isDivisible = [](int x, int y) {
    return x % y == 0;
};
```

Example Binary Operations

```
// returns the sum of its arguments
auto sum = [](double x, double y) {
    return x + y;
};

// removes a substring from a string, if the substring
```

```

// exists. otherwise, it returns an empty string
auto removeSubstring =
[](const std::string& str, const std::string& substr) {
    std::size_t substrStart = str.find(substr);

    if (substrStart != std::string::npos) {
        std::string copy = str;
        copy.erase(substrStart, substr.length());

        return copy;
    }
    // explicitly calling string constructor so the
    // return type does not need to be specified
    else return std::string("");
};

```

File I/O

Reading and writing to files is the primary task for most computer programs. C++ provides the `fstream` header to read and write files. The `std::ifstream` class is used to read files and the `std::ofstream` class is used for writing files. These input and output streams provide the same operations as `std::cin` and `std::cout`, respectively. The difference is you first have to open a file for reading or writing.

Reading Files

To read a file, you need to declare an `std::ifstream` object. Files can be opened by passing a file path to the `ifstream` constructor, or by calling the `open()` function:

```

#include <fstream>
#include <iostream>
#include <string>

// reading file path from command line args
int main(int argc, char** argv) {
    if (argc < 2) {
        std::cerr << "Specify input file path as command line arg\n";
        return 1;
    }

    // open file with constructor
    std::ifstream fin(argv[1]);
    // alternative method:
    /*
    std::ifstream fin;

```

```

    fin.open(argv[1]);
    */

    // check that the file was opened correctly
    if (fin.is_open()) {
        // printing the contents of the file
        std::string input;
        // the >> operator works just like it does
        // for std::cin. it reads characters until
        // reaching a whitespace character
        // when reaching the end of the file, this
        // will return a "false-like" value
        while (fin >> input)
            std::cout << input << "\n";
    }
    else {
        std::cerr << "Failed to open file " << argv[1] << "\n";
        return 2;
    }

    // always close a file when you are done with it
    fin.close();

    return 0;
}

```

If we assume the contents of the file we are reading look like this:

```

To be, or not to be, that is the question.
2 + 2 = 4
Lorem ipsum dolor...

```

Then the output of this program would be:

```

To
be,
or
not
to
be,
that
is
the
question.
2
+
2
=

```

```
4
Lorem
ipsum
dolor...
```

Like `cin`, `ifstream` objects can also directly read numbers and other data types from a file. Let's assume we have a file that looks like this:

```
2 2
4 4
6 6
8 8
```

We want to write a program that reads in the pairs of numbers and prints the result to the console.

```
#include <fstream>
#include <iostream>

// filestreams always need to be passed by reference
int readAndAdd(std::ifstream& fin) {
    int x;
    int y;
    int sum = 0;
    if (fin >> x >> y) {
        sum = x + y;
    }
    return sum;
}

int main(int argc, char** argv) {
    if (argc < 2) {
        std::cerr << "Specify input file path as command line arg\n";
        return 1;
    }

    // open file with constructor
    std::ifstream fin(argv[1]);

    // alternate method to check if file is open:
    if (fin.fail()) {
        std::cerr << "Failed to open file " << argv[1] << "\n";
        return 2;
    }

    // run the function readAndAdd() until the file is empty
    // fin.eof() returns true when the end of the file is reached
    while (!fin.eof()) {
```

```

        int sum = readAndAdd(fin);
        std::cout << "Sum = " << sum << "\n";
    }

    // close file
    fin.close();

    return 0;
}

```

There is the `std::getline()` function, which can read an entire line from a file into a string, preserving the whitespace. The first argument to `std::getline()` is an `ifstream` object, and the second argument is the `std::string` the line will be stored in.

Writing Files

Creating and checking if an `std::ofstream` object is open is the same as it is for input files. Like `cout`, `ofstream` uses the insertion operator (`<<`) to write data to files. By default, output file streams truncate (i.e., delete) the contents of a file before writing to it. If the file does not exist, it will create the file. If you want to add data to an existing file, the `ios::app` option can be specified when opening the file. This flag is an option argument to the constructor and `open()` function. Below is an example program that copies one files contents to another. Here, we are using another member function of `ifstream`, `get()`, which reads a single character at a time:

```

#include <fstream>
#include <iostream>

int main(int argc, char** argv) {
    const char* errmsg = "Invalid usage. Expected 2 arguments\n";
    const char* help = "Usage: ./copy input_file output_file\n";

    if (argc < 3) {
        std::cerr << errmsg;
        std::cerr << help;
    }

    std::ifstream fin(argv[1]);
    if (fin.fail()) {
        std::cerr << "Failed to open file " << argv[1] << "\n";
        return 2;
    }
}

```

```

std::ofstream fout(argv[2]);
if (fin.fail()) {
    // close fin before ending the program
    fin.close();
    std::cerr << "Failed to open file " << argv[2] << "\n";
    return 3;
}

char c;
// while we are not at eof and can read a character
while (!fin.eof() && fin.get(c)) {
    // print that character to the output file
    // this will create a character for character copy
    // of the input file, since get() does not skip whitespace
    fout << c;
}

fin.close();
fout.close();
}

```

Conclusion

This week, we discussed the three mechanisms for callback functions in C++. First, we looked at function pointers. Function pointers have a unique syntax for the parameter of the function that expects a call back, but any function can be passed as an argument to become a function pointer. The function pointer parameter syntax has largely been supplanted by the `std::function` type in modern C++.

`std::function` also allows functors and lambdas to be passed. A functor (also called a function object), is a class or struct that implements `operator()`, making objects of that class (or struct) callable like they are functions. Functors are useful in cases where data needs to be maintained with the callable function. The final type of callback is lambda expressions. Lambdas were introduced in C++11, and permit inline function declarations as expressions. Finally, we discussed some terminology that will be used to describe callback functions as they relate to their usage with the STL algorithms. Afterwards, a quick overview of reading and writing files was provided.